

Definições de Recursos Personalizados (Custom Resource Definitions)

O que você aprenderá nesta página

Como criar as suas próprias Definições de Recursos Personalizados (CRDs - *Custom Resource Definitions*)

As Definições de Recursos Personalizados (CRDs) são uma forma de estender o Kubernetes com nossos próprios Recursos. Já utilizamos uma grande quantidade deles; por exemplo, utilizamos a CRD **Application** com o ArgoCD. Eles são uma parte tão fundamental na utilização do Kubernetes que é uma boa ideia aprender a criar os nossos próprios.

Antes de começarmos, precisamos decidir o que queremos criar. Então, vamos criar um recurso que pode ser usado para configurar contagens regressivas (*countdowns*). O recurso se chamará "Countdown". Ele terá uma certa duração (*length*) e um intervalo (*delay*) entre as execuções. A execução – o que acontece cada vez que o *delay* transcorre – fica a cargo de uma imagem. Por exemplo, alguém que usar nossa CRD poderá criar um cronômetro que poste uma mensagem no Twitter toda vez que uma contagem decrescer.

Como modelo, usarei um disponibilizado na documentação oficial, portanto, vamos colar o seguinte código no arquivo *resourcedefinition.yaml*:

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  # o nome deve corresponder aos campos spec abaixo, e ter o formato: <plural>.<group>
  name: countdowns.stable.dwk
spec:
  # nome do grupo para uso da API REST: /apis/<group>/<version>
  group: stable.dwk
  # as opcoes podem ser Namespaced ou Cluster
  scope: Namespaced
  names:
    # kind (tipo) normalmente e escrito no formato CamelCased e no singular.
    kind: Countdown
    # nome plural usado na URL: /apis/<group>/<version>/<plural>
    plural: countdowns
    # nome singular usado como atalho (alias) na linha de comando (CLI)
    singular: countdown
    # nomes curtos (shortNames) permitem buscar seu recurso de forma abreviada no CLI
    shortNames:
      - cd
  # lista das versoes suportadas por essa CustomResourceDefinition
  versions:
    - name: v1
      # Cada versao pode ser ligada/desligada por esta flag
      served: true
      # Apenas uma unica versao deve ser marcada como storage
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                length:
                  type: integer
                delay:
                  type: integer
                image:
```

```

        type: string
additionalPrinterColumns:
- name: Length
  type: integer
  description: A duracao (length) do countdown
  jsonPath: .spec.length
- name: Delay
  type: integer
  description: 0 intervalo de tempo (delay, em ms) entre as execucoes
  jsonPath: .spec.delay

```

Agora, podemos instanciar nosso próprio Countdown no arquivo *countdown.yaml*:

```

apiVersion: stable.dwk/v1
kind: Countdown
metadata:
  name: doomsday
spec:
  length: 20
  delay: 1200
  image: jakousa/dwk-app10:sha-84d581d

```

E, em seguida, aplica-lo:

```

$ kubectl apply -f countdown.yaml
countdown.stable.dwk/doomsday created

$ kubectl get cd
NAME          LENGTH  DELAY
doomsday      20      1200

```

Temos agora um novo recurso! Em seguida, vamos criar um controlador personalizado (*custom controller*) que vai inicializar um *pod* rodando o contêiner gerado a partir da imagem, e vai garantir que os "countdowns" sejam destruídos após o fim. Isso exigirá um pouco de codificação.

Para a implementação, escolhi utilizar o recurso **Job** (*tarefa*). Contêineres rodados através de Jobs são projetados para rodarem apenas uma vez até o fim (*completion*). Contudo, nem os Jobs concluídos e nem os *Pods* são excluídos sozinhos, sendo mantidos apenas para que os logs de execução possam ser revisados posteriormente.

Nosso controlador precisa realizar 3 ações:

Criar um *Job* a partir de um *Countdown*. Reagendar os *Jobs* até que a quantidade de execuções definidas no *Countdown* (*length*) seja completada. Limpar todos os *Jobs* e *Pods* logo após as execuções.

Para implementar o controlador, precisamos executar certas interações de "baixo nível" e acessar o Kubernetes diretamente usando as APIs REST.

Observando (escutando) a API do Kubernetes no endereço `/apis/stable.dwk/v1/countdowns?watch=true`, receberemos um status `ADDED` para cada novo objeto *Countdown* criado no cluster. Feito isso, criar a tarefa consiste em compilar a mensagem contendo as propriedades JSON e realizar um `POST` com a carga de dados (payload) válida para `/apis/batch/v1/namespaces//jobs`.

No que se refere às tarefas (*Jobs*), ficaremos monitorando `/apis/batch/v1/jobs?watch=true` e esperaremos pelo evento `MODIFIED` no qual o estado de sucesso seja igual a *true*, e atualizaremos os rótulos (*labels*) dos *jobs* para armazenar seus respectivos *status*. Para apagar um *Job* e seu respectivo *Pod*, basta mandar requisições "delete" para `/api/v1/namespaces//pods/` e para `/apis/batch/v1/namespaces//jobs/`.

Finalmente, é só realizar uma chamada "delete" para `/apis/stable.dwk/v1/namespaces//countdowns/` a fim de deletar o *countdown*.

RBAC

RBAC (*Role-based access control*, ou Controle de Acesso Baseado em Funções) é uma metodologia de autorização que nos permite definir permissões de acesso para diferentes usuários, *service accounts* e/ou grupos por meio da concessão de funções (*roles*). Para nosso caso de uso, definiremos um *ServiceAccount* no arquivo *serviceaccount.yaml*:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: countdown-controller-account
```

Em seguida, informaremos a *serviceAccountName* que a nossa aplicação utilizará em seu *deployment.yaml*:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: countdown-controller-dep
spec:
  replicas: 1
  selector:
    matchLabels:
      app: countdown-controller
  template:
    metadata:
      labels:
        app: countdown-controller
    spec:
      serviceAccountName: countdown-controller-account
      containers:
        - name: countdown-controller
          image: jakousa/dwk-app10-controller:sha-4256579
```

A etapa seguinte é definir as permissões necessárias e agrupá-las em "Funções" (*roles*). O Kubernetes reconhece dois tipos de *roles*: o *ClusterRole* e o *Role*. Enquanto a permissão atribuída a um *Role* é restrita aos limites de um único namespace (*namespace-specific*), o *ClusterRole* abrange todo o cluster.

As permissões das regras (*rules*) são parametrizadas a partir da união de um grupo (*apiGroup*), do alvo principal (*resource*), bem como do verbo ou dos verbos (*verbs* - Get, Post, Watch, List etc) vinculados a eles.

A API no caso é `/apis/batch/v1/jobs?watch=true`, o *apiGroup* será o "batch", o *resource* as "jobs", e os verbos a lista daquilo que queremos autorizar. Quando o *apiGroup* estiver vazio (""), isso sinaliza ao Kubernetes que os recursos em questão vêm diretamente das APIs centrais. As definições ficarão parecidas com estas em um arquivo *clusterrole.yaml*:

```
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: countdown-controller-role
rules:
- apiGroups: [""]
  # no nível HTTP, o nome do recurso para acessar Pods e "pods"
  resources: ["pods"]
  verbs: ["get", "list", "delete"]
- apiGroups: ["batch"]
  # no nível HTTP, o nome do recurso para acessar Jobs e "jobs"
  resources: ["jobs"]
  verbs: ["get", "list", "watch", "create", "delete"]
- apiGroups: ["stable.dwk"]
  resources: ["countdowns"]
  verbs: ["get", "list", "watch", "create", "delete"]
```

E por último, conectaremos a "Role" recém declarada na nossa "ServiceAccount". Assim como nos tipos de Funções (*roles*), a *RoleBinding* limita-se a um único namespace, e uma *ClusterRoleBinding* é transversal e se ramificará por toda a estrutura do Kubernetes.

No nosso projeto, utilizaremos o `ClusterRoleBinding` para garantir acesso a todos os namespaces disponíveis. Segue o código (*clusterrolebinding.yaml*):

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: countdown-rolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: countdown-controller-role
subjects:
- kind: ServiceAccount
  name: countdown-controller-account
  namespace: default
```

Precisamos de um recurso que extraia automaticamente links HTML como conteúdo (vamos dar o nome de *DummySite*) e gere uma cópia de sites baseada nas urls fornecidas.

Crie a propriedade `website_url` dentro de sua nova *CustomResourceDefinition*. Crie o "controller" que obterá e analisará todos os *DummySites* criados a partir das APIS REST do Kubernetes. Faça o controlador gerar e preencher as requisições *Job* de que seu código precisará para concretizar os alvos.